

category: avalonia

tags: [writeablebitmap, memory-leak, dispose, bitmap-binding, native-buffer]

severity: high

created: 2026-07-23

updated: 2026-07-23

project-origin: iPGSv4 (IPGS.RemoteControl.CcuUI)

WriteableBitmap gán lại vào binding property mà không Dispose cái cũ → rò rỉ native buffer

Tình huống gặp phải

Đang review CCU-side của tính năng Remote Control (CCU → ZCU) trong iPGSv4. ViewModel nhận stream JPEG 10–15 fps từ ZCU, decode ra `SKBitmap` rồi tạo `WriteableBitmap` mới cho mỗi frame và gán vào `CurrentFrame` (property có `[ObservableProperty]` để bind vào `Image.Source` trên UI).

Triệu chứng / Lỗi

- Bộ nhớ tiến trình tăng nhanh sau vài giây streaming (mỗi frame 1080p BGRA ≈ 8 MB).
- GC không kịp thu hồi vì `WriteableBitmap` giữ buffer pixel ở NATIVE (unmanaged).
- Không có exception – chỉ RAM leo lên đều, dễ bỏ sót khi test nhanh.

Nguyên nhân gốc rễ (Root Cause)

Avalonia `WriteableBitmap` wrap 1 pixel buffer unmanaged. Khi property `CurrentFrame` được gán bitmap mới:

- Reference tới bitmap cũ bị mất → managed GC sẽ dọn (finalizer đóng buffer).
- Nhưng **finalizer chạy khi GC muốn**, không ngay lập tức. Ở tốc độ $15 \text{ fps} \times 8 \text{ MB} = \sim 120 \text{ MB/s}$ allocation, GC gen2 không kịp và bộ nhớ native tăng gấp bội trước khi finalizer chạy.

Đây là pattern chung của MỌI wrapper .NET quanh native resource (Bitmap, Stream, Font, GDI handles, ...): **phải Dispose thủ công**, không phụ thuộc GC.

Giải pháp

```
// SAI (leak):
Dispatcher.UIThread.Post(() => CurrentFrame = wb);

// ĐÚNG:
Dispatcher.UIThread.Post(() =>
{
    var old = CurrentFrame;
    CurrentFrame = wb;    // binding cập nhật trước
    old?.Dispose();       // rồi dispose bản cũ trên UI thread
});
```

Và trong `Dispose()` của ViewModel:

```
CurrentFrame?.Dispose();  
CurrentFrame = null;
```

Áp dụng lại (How to reuse)

- Bất kỳ property binding kiểu `Bitmap/WriteableBitmap/RenderTargetBitmap` **được gán lại nhiều lần** → **PHẢI** dispose bản cũ khi gán bản mới.
- Luôn dispose trên **UI thread** (dùng `Dispatcher.UIThread.Post`) để tránh race với render layer đang dùng bitmap đó vẽ frame.
- Trong `IDisposable.Dispose()` của owner (ViewModel/Window), nhớ dispose luôn bitmap cuối cùng — không để lại đến finalizer.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ **KHÔNG** dispose bitmap cũ **TRƯỚC** khi gán bản mới trên UI thread — nếu render layer chưa flip sang bitmap mới, có thể vẽ vào buffer đã giải phóng → crash native. Thứ tự đúng: gán mới → dispose cũ (cùng UI thread, tuần tự).
- ⚠ Đừng tin "GC sẽ lo" cho bất cứ wrapper native nào. Managed heap có thể còn thoáng trong khi native buffer đã cạn RAM.
- ⚠ Tối ưu double-buffer (giữ 2 `WriteableBitmap` luân phiên, chỉ update pixel không allocate mới) là bước v1.1 nếu profile thấy GC pressure còn cao — nhưng phải fix leak trước.

Tham chiếu

- File fix: `IPGS.RemoteControl.CcuUI/ViewModels/RemoteScreenViewModel.cs` (OnFrameReceived + Dispose)
- Avalonia docs: `Avalonia.Media.Imaging.Bitmap` implements `IDisposable`
- Project liên quan: `iPGSv4 / IPGS.RemoteControl.CcuUI`